hashlock.

Security Audit

Klima Protocol “Main” (DeFi)

December 2025

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

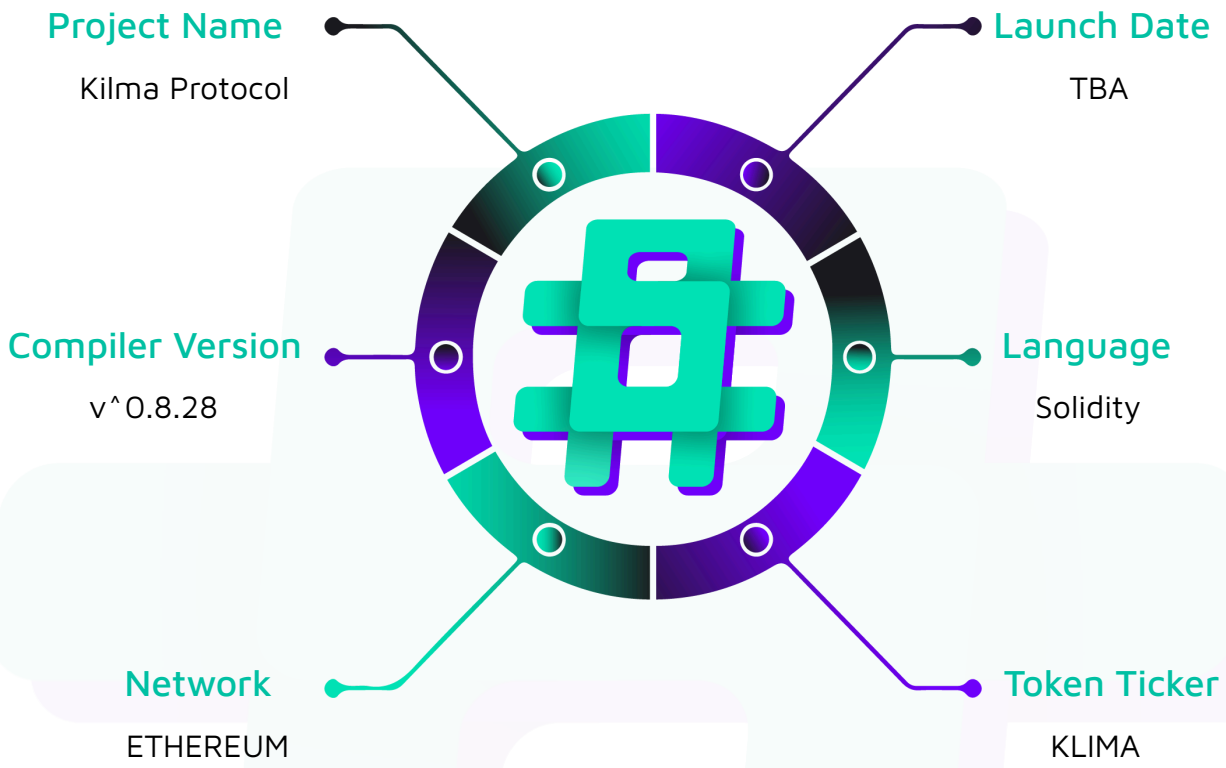
The Klima Protocol team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Klima Protocol is a Web3 / DeFi-based climate-finance protocol and DAO that brings carbon markets on-chain: it tokenizes carbon credits as digital assets, and issues a native token KLIMA — each backed by at least one carbon tonne in the protocol's treasury. Through smart-contracts on the Polygon blockchain, Klima protocol provides liquidity, enables users to buy/retire carbon credits, stake or bond KLIMA, and participate in decentralized governance — with the stated mission of making carbon markets transparent, liquid, and accessible globally, and channeling more capital toward climate impact and carbon offset projects.

Project Name: Kilma Protocol
Project Type: DeFi, Token
Compiler Version: ^0.8.28
Website: https://www.klimaprotocol.com/
Logo:

Visualised Context:



Audit Scope

We at Hashlock audited the solidity code within the KilmaDAO project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**KilmaDAO Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**December, 2025**
GitHub Handle	https://github.com/KlimaDAO/klima-v2/tree/main
Contract 1	contracts/CarbonLedger.sol
Contract 2	contracts/CarbonClassVault.sol
Contract 3	contracts/CarbonClassCoupons.sol
Contract 4	contracts/ClassVaultRegistrationManager.sol
Contract 5	contracts/CarbonVaultFactory.sol
Contract 6	contracts/MaturityManager.sol
Contract 7	contracts/ProtocolAccessManager.sol
Contract 8	contracts/ProtocolMinter.sol
Contract 9	contracts/ProtocolRewardsEscrow.sol
Contract 10	contracts/ProtocolSupplyOracle.sol
Contract 11	tokens/K2.sol
Contract 12	tokens/KVCM.sol
Contract 13	tokens/KvcmLockNFT.sol
Contract 14	diamonds/AssetManager.sol
Contract 15	diamonds/RewardManager.sol
Contract 16	diamonds/StakingManager.sol

Contract 17	facets/AssetManagerConfigFacet.sol
Contract 18	facets/OperationsFacet.sol
Contract 19	facets/RateLimiterFacet.sol
Contract 20	facets/BaseCarbonClassQuoter.sol
Contract 21	facets/SampleCarbonClassQuoter.sol
Contract 22	facets/QuoterManagerFacet.sol
Contract 23	facets/QuoterRouterFacet.sol
Contract 24	core/DiamondCutFacet.sol
Contract 25	core/DiamondLoupeFacet.sol
Contract 26	core/OwershipFacet.sol
Contract 27	reward-manager/K2StakersRewardsFacet.sol
Contract 28	reward-manager/K2YieldCurveFacet.sol
Contract 29	reward-manager/K2YieldFacet.sol
Contract 30	reward-manager/KvcmIncentivesCurveFacet.sol
Contract 31	reward-manager/RMKVCMStakersRewardsFacet.sol
Contract 32	reward-manager/RMLPRewardsFacet.sol
Contract 33	reward-manager/RewardManagerConfigFacet.sol
Contract 34	reward-manager/RewardManagerViewFacet.sol
Contract 35	reward-manager/RiskyYieldFacet.sol
Contract 36	reward-manager/RollUpdateFacet.sol
Contract 37	staking-manager/BaseLPStakingFacet.sol
Contract 38	staking-manager/K2StakingFacet.sol
Contract 39	staking-manager/KvcmLPStakingFacet.sol
Contract 40	staking-manager/KvcmStakingFacet.sol
Contract 41	staking-manager/StakingManagerConfigFacet.sol
Contract 42	staking-manager/StakingManagerPauseFacet.sol
Contract 43	staking-manager/StakingManagerViewFacet.sol

Contract 44	pairs/KvcmK2LPFacet.sol
Contract 45	pairs/KvcmUSDCLPFacet.sol
Contract 46	interfaces/IAssetManager.sol
Contract 47	interfaces/IAssetManagerConfigFacet.sol
Contract 48	interfaces/ICarbonClassQuoter.sol
Contract 49	interfaces/IOperationsFacet.sol
Contract 50	interfaces/IQuoterManager.sol
Contract 51	interfaces/IQuoterRouterFacet.sol
Contract 52	interfaces/IRateLimiterFacet.sol
Contract 53	RewardManager/IK2StakersRewardsFacet.sol
Contract 54	RewardManager/IK2YieldCurveFacet.sol
Contract 55	RewardManager/IK2YieldFacet.sol
Contract 56	RewardManager/IKvcmIncentivesCurveFacet.sol
Contract 57	RewardManager/IRewardManager.sol
Contract 58	RewardManager/IRewardManagerConfigFacet.sol
Contract 59	RewardManager/IRewardManagerViewFacet.sol
Contract 60	RewardManager/IRiskyYieldFacet.sol
Contract 61	RewardManager/IRollUpdateFacet.sol
Contract 62	StakingManager/IK2StakingFacet.sol
Contract 63	StakingManager/IK2StakingFacet.sol
Contract 64	StakingManager/IKvcmLPStakingFacet.sol
Contract 65	StakingManager/IKvcmLock.sol
Contract 66	StakingManager/IKvcmStakingFacet.sol
Contract 67	StakingManager/IStakingManager.sol
Contract 68	StakingManager/IStakingManagerConfigFacet.sol
Contract 69	StakingManager/IStakingManagerViewFacet.sol
Contract 70	Tokens/ICarbonClassCoupons.sol

Contract 71	Tokens/ICarbonClassVault.sol
Contract 72	Tokens/ICarbonLedger.sol
Contract 73	Tokens/ICarbonVaultFactory.sol
Contract 74	Tokens/ICommonEvents.sol
Contract 75	Tokens/IDiamondCut.sol
Contract 76	Tokens/IDiamondLoupe.sol
Contract 77	Tokens/IERC173.sol
Contract 78	Tokens/IMaturityManager.sol
Contract 79	Tokens/IPool.sol
Contract 80	Tokens/IProtocolMinter.sol
Contract 81	Tokens/IProtocolRewardsEscrow.sol
Contract 82	Tokens/IProtocolSupplyOracle.sol
Contract 83	Tokens/IRetirementAggregator.sol
Contract 84	Tokens/IRewardsEscrow.sol
Contract 85	Tokens/IYieldCurveFacet.sol
Contract 86	libraries/AAMConstants.sol
Contract 87	libraries/AAMUtils.sol
Contract 88	libraries/OperationsMath.sol
Contract 89	libraries/RateLimiterLisb.sol
Contract 90	libraries/K2YieldUtilsLib.sol
Contract 91	libraries/KvcmIncentivesCurveMathLib.sol
Contract 92	libraries/RiskyYieldUtilsLib.sol
Contract 93	libraries/AssetManagerStorage.sol
Contract 94	libraries/CarbonVaultStorage.sol
Contract 95	libraries/DiscountFactorStorage64.sol
Contract 96	libraries/RewardManagerStorage.sol
Contract 97	libraries/StakingManagerStorage.sol

Contract 98	libraries/AeroLPUtils.sol
Contract 99	libraries/CarbonLedgerStorage.sol
Contract 100	libraries/CarbonSystemRoleLib.sol
Contract 101	libraries/CarbonTonnageLib.sol
Contract 102	libraries/CarbonVaultStorage.sol
Contract 103	libraries/CarbonVaultSynchronizerLib.sol
Contract 104	libraries/ClassVaultRegistrationUtilsLib.sol
Contract 105	libraries/CommonStructs.sol
Contract 106	libraries/Constants.sol
Contract 107	libraries/Errors.sol
Contract 108	libraries/LibDiamond.sol
Contract 109	libraries/LibTransfer.sol
Contract 110	libraries/LibUtils.sol
Contract 111	libraries/RevertHelper.sol
Contract 112	libraries/ValidationLib.sol
Audited GitHub Commit Hash	e92b306756f273b62d2b3f4004ef559bc2f09c37
Fix Review GitHub Commit Hash	65ae70a00fe44966a7263a469959e0b6059ca334

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

6 High severity vulnerabilities

22 Medium severity vulnerabilities

7 Low severity vulnerabilities

9 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
CarbonClassCoupons.sol ERC20 token contract representing carbon tonnes in a CarbonClassVault. Provides functionality to issue and burn coupons that represent reserved carbon tonnes. Coupons can be issued by authorized roles (coupons issuer, admin, or liquidity manager) and burned to release reserved tonnes. Tracks reserved tonnes separately from total tonnes to ensure available tonnes calculations.	**Contract achieves this functionality.**
CarbonClassVault.sol Upgradeable vault contract for storing carbon credit tokens (ERC20 and ERC1155) for a specific carbon class. Tracks total equivalent carbon tonnes represented by deposited tokens, manages deposits and withdrawals with maturity tracking, handles liquid vs forward carbon accounting, and includes a coupon system for tokenizing carbon tonnes. Supports both ERC20 and ERC1155 carbon credit tokens with conversion between token amounts and tonnes.	**Contract achieves this functionality.**
CarbonLedger.sol Central registry contract managing carbon class vaults and their associated carbon credit tokens. Handles registration/unregistration of class vaults and carbon credits, manages credit configurations (token type, conversion rates), provides liquidity management functions for adding/removing credits to/from vaults, supports migration of credits between class vaults, and	**Contract achieves this functionality.**

manages forward carbon maturation.	
CarbonVaultFactory.sol Factory contract for deploying and managing CarbonClassVault proxy instances. Manages implementation versions, deploys new vault proxies with specified configurations, tracks deployed vaults, handles vault upgrades to new implementation versions, and manages vault active status.	**Contract achieves this functionality.**
ClassVaultRegistrationManager.sol Contract designed to be used via delegatecall from CarbonClassVault for managing carbon credit token registration. Handles registration and unregistration of ERC20 and ERC1155 tokens within vaults, maintains token registries with efficient storage patterns, and validates token configurations.	**Contract achieves this functionality.**
MaturityManager.sol Manages lock maturities with deterministic mapping between timestamps and maturity IDs. Defines 90-day maturity periods, tracks active maturity ranges (40 active maturities), provides maturity timestamp calculations, validates maturity IDs, and manages midnight indexing for day-based calculations.	**Contract achieves this functionality.**
ProtocolAccessManager.sol Manages role-based permissions for the entire system using OpenZeppelin's AccessManager pattern. Provides centralized access control for all protocol contracts.	**Contract achieves this functionality.**
ProtocolMinter.sol Central minting authority for the protocol. Mints KVCM tokens to authorized contracts (RewardManager and AAM), transfers K2 tokens from RewardsEscrow to	**Contract achieves this functionality.**

RewardManager, and manages configuration of protocol token addresses and contract references.	
ProtocolRewardsEscrow.sol Escrow contract for managing K2 rewards distribution. Escrows K2 tokens from ProtocolMinter, transfers K2 to Treasury (admin-controlled), transfers K2 to RewardManager (for distribution), and enforces escrow limits.	**Contract achieves this functionality.**
ProtocolSupplyOracle.sol Oracle contract providing effective supply calculations for KVCM and K2 tokens. Used by other protocol contracts for supply dependent calculations.	**Contract achieves this functionality.**
BaseLpStakingCore.sol Abstract base contract for LP token staking functionality. Provides core staking/unstaking logic for LP tokens with maturity-based locking, validates maturity IDs through MaturityManager, tracks global and maturity-specific staking totals, and supports permit-based approvals. Extended by KvcmK2LpModule and KvcmUSDCLpModule specific modules.	**Contract achieves this functionality.**
K2.sol Upgradeable ERC20 token with capped supply, mint/burn capabilities, permit functionality, and role based access control. Used as a protocol reward token with maximum supply cap.	**Contract achieves this functionality.**
KVCM.sol Upgradeable ERC20 token with mint/burn capabilities, permit functionality, and role-based access control.	**Contract achieves this functionality.**
KvcmLockNFT.sol	**Contract achieves this**

Upgradeable ERC721 NFT contract for representing KVCM locks. Provides restricted transfer functionality (only allowed by authorized managers), minting/burning by liquidity managers, and token URI storage for metadata.	**functionality.**
AssetManager.sol Diamond contract for asset management operations. Functionality provided through facets: - DiamondCutFacet: Manages adding, replacing, removing facets and functions. - DiamondLoupeFacet: Provides introspection functions to query facets and function selectors - OwnershipFacet: Manages diamond ownership and transfer - AssetManagerConfigFacet: Manages global configuration (access manager, protocol contracts, tokens) - OperationsFacet: Handles core asset management operations (deposits, withdrawals, retirements) - QuoterRouterFacet: Routes quote requests to appropriate quoters for different carbon classes - RateLimiterFacet: Implements rate limiting for operations	**Contract achieves this functionality.**
RewardManager.sol Diamond contract for reward distribution and management. Functionality provided through facets: - DiamondCutFacet: Manages adding, replacing,	**Contract achieves this functionality.**

removing facets and functions. - DiamondLoupeFacet: Provides introspection functions to query facets and function selectors - OwnershipFacet: Manages diamond ownership and transfer - RewardManagerConfigFacet: Manages global configuration - RewardManagerViewFacet: Provides view functions for reward data - K2StakersRewardsFacet: Manages K2 rewards for stakers - K2YieldFacet: Handles K2 yield calculations and distribution - K2YieldCurveFacet: Manages K2 yield curve parameters - KvcmIncentivesCurveFacet: Manages KVCM incentives curve for stakers - RMKVCMStakersRewardsFacet: Manages KVCM rewards for stakers - RMLPRewardsFacet: Manages rewards for LP token stakers - RiskyYieldFacet: Handles risky yield calculations - RollUpdateFacet: Manages rolling updates for reward curves and distributions	
StakingManager.sol Diamond contract for staking operations. Functionality provided through facets: - DiamondCutFacet: Manages adding, replacing, removing facets and functions. - DiamondLoupeFacet: Provides introspection functions to query facets and function selectors	**Contract achieves this functionality.**

- OwnershipFacet: Manages diamond ownership and transfer - StakingManagerConfigFacet: Manages global configuration - StakingManagerViewFacet: Provides view functions for staking data - StakingManagerPauseFacet: Manages pause states (system-wide and per-staking-type) - KvcmStakingFacet: Handles KVCM token staking with maturity-based locks - K2StakingFacet: Handles K2 token staking (unlocked staking) - KvcmLPStakingFacet: Main entry point for LP token staking, delegates to pair-specific modules	

Code Quality

This audit scope involves the smart contracts of the KilmaDAO project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the KilmaDAO project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] K2StakingFacet.sol#_allocateK2 - Incorrect beta calculation due to passing total instead of delta in k2 allocation

Description

The `_allocateK2()` function incorrectly passes the new total K2 allocations `newTotalK2Allocations` to `IRiskyYieldFacet.updateBetaSqForK2Allocation()` instead of the delta amount `newAllocationAmount`. This causes incorrect beta calculations leading to incorrect reward distributions.

Vulnerability Details

In `_allocateK2()`, the function calculates `newTotalK2Allocations` and then incorrectly passes this total value to `updateBetaSqForK2Allocation()`:

```
uint256 newTotalK2Allocations = l.totalK2Allocations + newAllocationAmount;

// Now update storage

l.allocationsByCarbonClass[carbonClass] += newAllocationAmount;

l.totalK2Allocations = newTotalK2Allocations;

uint64                                  currentDayId                                    =
SafeCast.toUint64(IMaturityManager(config.maturityManager).getMidnightIndexForTimestamp(b
lock.timestamp));

// Update beta and lambdas with the NEW total

IRiskyYieldFacet(config.rewardManager).updateBetaSqForK2Allocation(

    newTotalK2Allocations, carbonClass, currentDayId

);
```

However, `_updateBetaSqForK2Action()` expects a delta value (the amount being allocated), not a total.

This inconsistency is confirmed by examining other functions in the codebase: `_deallocateK2()` correctly passes the delta `deallocateAmount`, `_reallocateK2()` correctly passes the delta amount for both deallocation and allocation, `KvcmStakingFacet` consistently passes the delta amount for both allocation and deallocation operations.

When `newTotalK2Allocations` is passed instead of `newAllocationAmount`, the beta calculation becomes incorrect because the factor calculation uses an incorrect value, the cache update accumulates totals instead of deltas and subsequent allocations will compound this error.

Impact

Incorrect beta squared calculations affects the beta cache state and lead to incorrect reward distributions.

Recommendation

Change the call to `updateBetaSqForK2Allocation()` to pass `newAllocationAmount` instead of `newTotalK2Allocations`.

Status

Resolved

[H-02] CarbonClassVault.sol#_depositCredits, _withdrawCarbonCredits - User controlled couponBurn parameter allows arbitrary coupon burn

Description

`couponBurn.from` is never tied to the address providing or receiving tokens during deposits or withdrawals, allowing externally supplied burn params to target arbitrary holders.

Vulnerability Details

`CarbonClassVault._depositCredits()` burns coupons from `couponBurn.from` but does not relate this address to the depositor `from` nor perform any caller authorization.

`OperationsFacet.swapCarbonCreditForKvcm()` accepts user controlled `couponBurnParams` and forwards them unchanged through `CarbonLedger.addCreditLiquidityToClassVault()` into `CarbonClassVault.depositCarbonCredits()`, which then calls `_burnCoupons(couponBurn.from, couponBurn.tonnes)`. Any user can supply an arbitrary address in `couponBurn.from` and burn that account's coupons, without the victim's consent.

```
// src/facets/aam/operations/OperationsFacet.sol

function swapCarbonCreditForKvcm(SwapCarbonCreditForKvcmParams memory swapParams) external {

    // ...

    ICarbonLedger(config.aamConfig.carbonLedger).addCreditLiquidityToClassVault(

        swapParams.carbonClass,

        swapParams.credit,

        swapParams.tokenId,

        swapParams.maturityId,

        msg.sender,

        rawTokenAmount,

        swapParams.couponBurnParams // user controlled
```

```
    );

    // ...

}
```

`CarbonClassVault._withdrawCarbonCredits()` repeats the same pattern, however the call is initiated from `OperationsFacet.retireCarbonCreditViaAggregator()` which is guarded by the `onlyRA` modifier.

Impact

An attacker can burn coupons from any holder and release their reserved tonnes, causing direct loss of coupon value for victims.

Recommendation

Validate that `couponBurn.from` matches the depositor `from` address. Also, make sure the `retirementAggregator` integration correctly handles the coupon burn parameters for the `retireCarbonCreditViaAggregator()` function, otherwise a similar validation is needed.

Status

Resolved

[H-03] KvcmStakingFacet#_unlockKvcm - Missing allocation cleanup on unlock

Description

The `_unlockKvcm()` function does not clean up allocation tracking structures when unlocking a matured KVCM lock, allowing users to inflate `allocationsByCarbonClass` by unlocking and re-locking the same funds, which directly impacts carbon pricing calculations used by the AAM.

Vulnerability Details

When a user unlocks a matured KVCM lock via `_unlockKvcm()`, the function does not clean up the allocation tracking structures.

The function contains a comment stating that cleanup is not necessary because "the midnight cron on maturity roll will remove allocations for that maturity." However, this assumption is not sufficient, as the midnight cron job only handles yield distribution and does not clean up `allocationsByCarbonClass`.

The `allocationsByCarbonClass` mapping is directly read by `StakingManagerViewFacet.getAllocationsForClass()` and used by the AAM for carbon pricing calculations in swap quotes. When `allocateKvcm()` is called, it increments `allocationsByCarbonClass[carbonClass]` and is only decremented when `deallocateKvcm()` is called.

This creates a scenario where the `allocationsByCarbonClass` mapping can be inflated:

1. User locks 1000 KVCM and allocates 1000 to CarbonClass A: `allocationsByCarbonClass[A] = 1000`
2. Lock matures, user calls `unlockKvcm()`, allocations remain: `allocationsByCarbonClass[A] = 1000`
3. User re-locks 1000 KVCM and allocates 1000 to CarbonClass A again → `allocationsByCarbonClass[A] = 2000` (should be `1000`)
4. The inflated allocation value is used in pricing calculations, leading to incorrect swap prices

Impact

Users can repeatedly unlock and re-lock the same KVCM tokens to artificially inflate `allocationsByCarbonClass`, causing incorrect carbon pricing calculations in the AAM swap mechanism, which can lead to economic loss through mispriced swaps.

Recommendation

The `_unlockKvcm()` function should validate that `lockInfo.priceAllocations == 0` before allowing the unlock operation. Forcing users to manually deallocate all allocations via `deallocateKvcm()` before unlocking the lock.

Status

Resolved

[H-04] RiskyYieldFacet.sol#_getBetaSqForDay - Incorrect default value causes underflow revert in deallocations

Description

The `_getBetaSqForDay()` function returns `0` when no previous betaSq value is found, but the comment indicates it should return `Constants.WAD` (1e18, representing beta of 1). This incorrect default value causes underflow reverts in deallocations.

Vulnerability Details

The `_getBetaSqForDay()` function returns the initial value of `0` when no previous value is found:

```
function _getBetaSqForDay(uint256 midnightIndex) internal view returns (uint256) {
    RewardManagerStorage.RMStorage storage r = RewardManagerStorage.rmStorage();
    // Check if betaSq exists for this midnightIndex
    uint256 betaSq = r.betaSqCache[midnightIndex];
    if (betaSq != 0) {
        // Rule 2: betaSq exists for this midnightIndex
        return betaSq;
    }
    // Rule 1: betaSq doesn't exist, use pointer to last non-zero betaSq
    uint256 lastIndex = r.lastNonZeroBetaSqIndex;
    if (lastIndex != 0 && lastIndex <= midnightIndex) {
        return r.betaSqCache[lastIndex];
    }
    // No previous value found - return initial value (1e18 = beta of 1)
    // TODO : Let's validate if this should be 0 or Constants.WAD
    return 0;
}
```

This function is used in `_updateBetaSqForKvcmAction()` and `_updateBetaSqForK2Action()` to get the previous betaSq value. In unstake operations, the calculation performs `x - y` where `x` is derived from `oldBetaSq`:

```
// Get previous betaSq value for this midnightIndex (uses _getBetaSqForDay helper)

uint256 oldBetaSq = _getBetaSqForDay(midnightIndex_); // scale -> e18

uint256 x = Math.mulDiv(oldBetaSq, _latestKvcmSupply, Constants.WAD);

// scale -> (e18 * e18 ) / e18 = e18

uint256 temp = Math.mulDiv(betaCache.k2Allocation, kvcmAllocation_, _latestK2Supply);

// scale -> (e18 * e18 ) / e18 = e18

uint256 y = Math.mulDiv(temp, (_latestK2Supply + betaCache.k2Allocation), _latestK2Supply);

if (stakeAction_ == StakeAction.Stake) {

    betaSq = Math.mulDiv(x + y, Constants.WAD, _latestKvcmSupply);

    betaCache.kvcmAllocation += kvcmAllocation_;

} else {

    betaSq = Math.mulDiv(x - y, Constants.WAD, _latestKvcmSupply);

    betaCache.kvcmAllocation -= kvcmAllocation_;

}
```

When `oldBetaSq` is `0`, `x` becomes `0` and so the initial `betaSq` value will be small, which will cause subsequent `betaSq` values to be small. Given that `x` is calculated using `oldBetaSq` and kvcm supply and `y` is calculated based on the kvcm and k2 supply and allocations, there could be the case that `x` is smaller than `y` and so the expression `x - y` will underflow, causing the transaction to revert and DoS the `KvcmStakingFacet.deallocateKvcm()` function. The same issue affects the `K2StakingFacet.deallocateK2()` function.

Proof of Concept

1. Remove the call `MockRiskyYieldFacet(address(rewardManager)).setBetaSq(1e18, 2);` in the `setUp()` function of the `StakingManager.Base.t.sol` file. So the function will use the default value of the function `_getBetaSqForDay()` instead of mocking it.

2. Add the following test to the `KvcmStaking.Deallocate.t.sol` file:

```
function test_underflow_deallocate() public {

    uint256 lockAmount = 0.1e18;

    uint256 allocationAmount = lockAmount;

    // Create lock without allocation first

    (uint256 start,) = getActiveMaturityRange();

    uint128 maturityId = SafeCast.toUint128(start);

    vm.startPrank(carbonWhale);

    kvcm.approve(address(stakingManager), lockAmount);

       uint256  lockId  =  IKvcmStakingFacet(address(stakingManager)).lockKvcm(lockAmount,
maturityId);

    vm.stopPrank();

    // Allocate

    vm.prank(carbonWhale);

                          IKvcmStakingFacet(address(stakingManager)).allocateKvcm(lockId,
address(carbonClassVault), allocationAmount);

    uint256 k2LockAmount = 1000 * 1e18;

    uint256 k2AllocationAmount1 = k2LockAmount;

    // Lock K2

    lockK2(carbonWhale, k2LockAmount);

    // Allocate K2

    vm.prank(carbonWhale);
```

```
                IK2StakingFacet(address(stakingManager)).allocateK2(k2AllocationAmount1,
address(carbonClassVault));

    // Deallocate Kvcm

    vm.prank(carbonWhale);

                        IKvcmStakingFacet(address(stakingManager)).deallocateKvcm(lockId,
address(carbonClassVault), allocationAmount); // This reverts with underflow

}
```

Run the test `forge test --match-test test_underflow_deallocate -vvv` which will revert with the following error: `[FAIL: panic: arithmetic underflow or overflow (0x11)] test_underflow_deallocate()`

Impact

Returning `0` instead of `Constants.WAD` causes underflow reverts, preventing users from deallocating kvcm and k2 tokens.

Recommendation

Change the return value from `0` to `Constants.WAD` in `_getBetaSqForDay()` when no previous value is found.

Status

Resolved

[H-05] OperationsFacet#retireCarbonCreditViaAggregator - Excess coupon burn on free retirement

Description

When a retirement is fully covered by coupons, the system correctly calculates a free retirement but burns the entire coupon amount instead of only the amount needed to cover the retirement.

Vulnerability Details

In `BaseCarbonClassQuoter.quoteRetirement()`, the function correctly identifies when coupons fully cover a retirement and returns a free retirement:

```
// if coupon covers (or over-covers) the whole retirement ⇒ free

if (couponBurnParams.tonnes >= tonnes) {

    // coupon covers (or over-covers) the whole retirement ⇒ free

                return    (tonnes,    0,    discountedForwardBalanceForCurrentMidnightIndex,
preTxnLiquidSupply);

}
```

However, the actual coupon burn logic in `OperationsFacet.retireCarbonCreditViaAggregator()` passes the full couponBurnParams to `ICarbonLedger.removeCreditLiquidityFromClassVault()` without adjusting the amount:

```
ICarbonLedger(config.aamConfig.carbonLedger).removeCreditLiquidityFromClassVault(

    retireParams.carbonClass,

    retireParams.credit,

    retireParams.tokenId,

    0,

    rawTokenAmount,

    config.aamConfig.retirementAggregator,

    retireParams.couponBurnParams
```

```
);
```

The `ICarbonLedger.removeCreditLiquidityFromClassVault()` function routes to `CarbonClassVault.withdrawCarbonCredits()`, which burns exactly the amount specified in `couponBurn.tonnes`, regardless of how much is actually needed for the retirement.

This creates a scenario where:

1. User wants to retire `tonnes = 8` tonnes
2. User sends `couponBurnParams.tonnes = 10` tonnes (to account for potential slippage)
3. System correctly calculates retirement is free (10 >= 8)
4. System incorrectly burns all 10 tonnes instead of just 8 tonnes
5. User loses 2 tonnes worth of coupons unnecessarily

Proof of Concept

Add the following test to the `OperationsFacet.Retire.ERC20.t.sol` file:

```
// Add the following imports:

import {IERC20} from "@openzeppelin/contracts/interfaces/IERC20.sol";

import {ICarbonClassVault} from "src/interfaces/ICarbonClassVault.sol";

import {QuoterRouterFacet} from "src/facets/aam/quotes/QuoterRouterFacet.sol";

import {IOperationsFacet} from "src/interfaces/AssetManager/IOperationsFacet.sol";

// ...  existing code ...

function test_retireCarbonCreditViaAggregator_burns_all_coupons_ERC20() public {

    TestToken memory t = testToken();

    TestCase memory tc = _setSingleRetirementTestCase(t);

    uint256 retirementTonnes = _toTonnes(tc.retirementAmount, t);

    uint256 extraTonnes = retirementTonnes * 2;

    vm.prank(KLIMA_ADMIN);

    carbonLedger.addCreditLiquidityToClassVault(
```

```
            address(carbonClassVault), t.credit, t.tokenId, 0, KLIMA_ADMIN, extraTonnes,
emptyCouponBurn()
    );
    uint256 availableTonnes = carbonClassVault.getAvailableTonnes();
       assertGt(availableTonnes, retirementTonnes * 2, "test needs excess tonnes for
overburn coupon");
      uint256 couponTonnes = retirementTonnes * 2; // more coupons than the retirement
needs
    issueCoupon(address(this), couponTonnes);
                                          uint256          couponBalanceBefore          =
IERC20(address(carbonClassVault)).balanceOf(address(this));
    ICarbonClassVault.CouponBurnParams memory couponBurnParamsNeeded =
                ICarbonClassVault.CouponBurnParams({tonnes: couponTonnes / 2, from:
address(this)});
    (, uint256 kvcmInNeeded,,) = QuoterRouterFacet(address(aam)).getRetirementQuote(
                address(carbonClassVault), t.credit, t.tokenId, tc.retirementAmount,
couponBurnParamsNeeded
    );
       assertEq(kvcmInNeeded, 0, "quote must be zero when coupons exceed retirement
amount");
    // Passing double of coupons needed
    ICarbonClassVault.CouponBurnParams memory couponBurnParams =
        ICarbonClassVault.CouponBurnParams({tonnes: couponTonnes, from: address(this)});
            IOperationsFacet.RetireCarbonCreditViaAggregatorParams memory params =
IOperationsFacet
        .RetireCarbonCreditViaAggregatorParams({
        carbonClass: address(carbonClassVault),
        credit: t.credit,
        tokenId: t.tokenId,
```

```
        amount: tc.retirementAmount,
        maxKvcmIn: 0,
        couponBurnParams: couponBurnParams
    });
    aggregatorPrank();
    IOperationsFacet(address(aam)).retireCarbonCreditViaAggregator(params);
                                        uint256          couponBalanceAfter          =
IERC20(address(carbonClassVault)).balanceOf(address(this));
    assertEq(couponBalanceAfter, 0, "all coupons burned");
}
```

Run the test and it should pass showing that all coupons are burned.

Impact

This results in loss of funds for users. When users send more coupons than strictly necessary to account for slippage or rounding (a common practice to ensure transactions succeed), the excess coupon amount is permanently burned.

Recommendation

Adjust the coupon burn amount to only burn the minimum needed amount. In `OperationsFacet.retireCarbonCreditViaAggregator()`, after receiving the quote, calculate the actual coupon burn amount needed.

Status

Resolved

[H-06] RMLPRewardsFacet#claimLPRewards - function is inaccessible to users due to incorrect access control

Description

The `claimLPRewards()` function is restricted by the `onlyStateUpdater` modifier, preventing users from claiming their own rewards. This contradicts the intended functionality and documentation, which implies users should be able to claim their rewards.

Vulnerability Details

The function applies the `onlyStateUpdater` modifier, which checks for the `REWARDS_MANAGER_STATE_UPDATER` role.

```
// RMLPRewardsFacet.sol

function claimLPRewards(address lpToken, uint256 maturityId) external onlyStateUpdater whenNotPaused {

    // ... logic for msg.sender claiming rewards }
```

The logic inside uses `msg.sender` as the beneficiary (`_settleYieldForLP(..., msg.sender)`), strongly indicating it is meant to be a user-facing function. However, the access control restricts it to a specific system role, making it impossible for a normal user to call it for themselves.

Impact

Users are unable to claim their LP rewards directly. Their funds (rewards) are effectively locked unless a centralized entity with the `onlyStateUpdater` role calls this function on their behalf (if the function logic even supports that, which it doesn't seem to, as it claims for `msg.sender`).

Recommendation

Remove the `onlyStateUpdater` modifier to allow any user to claim their own rewards.

Status

Resolved

Medium

[M-01] CarbonClassVault#updateCreditConfig - Credit config not synchronized with CarbonLedger

Description

When `CarbonLedger.updateCreditConfig` is called to update a credit token's configuration, the update is only applied to the `CarbonLedger`'s storage. CarbonClassVaults that already have the credit registered maintain stale configuration data, causing incorrect conversion calculations for deposits, withdrawals, and accounting operations.

Vulnerability Details

The `CarbonLedger.updateCreditConfig()` function updates the credit configuration only in the `CarbonLedger`'s storage:

```
function updateCreditConfig(
        address carbonCredit,
        CommonStructs.TokenType tokenType,
        CommonStructs.ConversionConfig memory conversionConfig
    ) external override onlyAdminOrManager whenNotPaused {
        // Check if credit exists before updating
        if (!_isCreditRegisteredInLedger(carbonCredit)) {
            revert Errors.CarbonCreditNotRegisteredInLedger(carbonCredit);
        }
        // Call internal function to set the info
        _setCreditConfig(carbonCredit, tokenType, conversionConfig);
    }
```

While `CarbonClassVault` has an `updateCreditConfig()` function that can be called by `CarbonLedger`, this function is never called by `CarbonLedger.updateCreditConfig()` or any other function from `CarbonLedger`, leaving vaults with outdated configuration.

When `CarbonLedger` updates a credit's conversion configuration, vaults continue using the old configuration values.

Impact

Incorrect conversion rates between tokens and tonnes cause accounting discrepancies, potentially leading to receiving incorrect amounts during deposits and withdrawals, and the system maintaining inaccurate tonne balances across all affected CarbonClassVaults.

Recommendation

Consider modifying `CarbonClassVault` to read credit configurations directly from `CarbonLedger` instead of storing its own copy. Vaults should call `ICarbonLedger(carbonLedger()).getCreditDetails(creditToken)` whenever they need the config for conversions. This eliminates the synchronization issue entirely and ensures vaults always use the most up to date configuration.

Status

Resolved

[M-02] CarbonLedger#migrateCreditBetweenClasses - Missing approval prevents migration from completing

Description

The `migrateCreditBetweenClasses()` function will always revert when attempting to deposit tokens to the target vault because `CarbonLedger` does not grant approval to the target vault before calling `depositCarbonCredits()`. The `_depositCredits()` function checks for approval from the from address (which is CarbonLedger in this case), but `CarbonLedger` never approves the target vault, causing the transaction to revert with `InsufficientApproval`.

Vulnerability Details

The `migrateCreditBetweenClasses()` function in `CarbonLedger` performs a two step migration process:

1. Withdraws tokens from the source vault to `CarbonLedger` (`address(this)`)
2. Deposits tokens from `CarbonLedger` to the target vault

```
// Migration flow:
// 1. Withdraw from Source Vault to this contract (CarbonLedger)
ICarbonClassVault(sourceVault).withdrawCarbonCredits(
    carbonCredit,
    0,
    0,
    amount,
    address(this),
    ICarbonClassVault.CouponBurnParams({tonnes: 0, from: address(0)})
);
// 2. Deposit to Target Vault
ICarbonClassVault(targetVault).depositCarbonCredits(
    carbonCredit,
```

```
    0,
    0,
    address(this),
    amount,
    ICarbonClassVault.CouponBurnParams({tonnes: 0, from: address(0)})
);
```

When `depositCarbonCredits()` is called, it internally calls `_depositCredits()` which performs approval checks before transferring tokens:

```
if (creditConfig.tokenType == CommonStructs.TokenType.ERC1155) {
    // Verify approval for ERC1155
    if (!IERC1155(creditToken).isApprovedForAll(from, address(this))) {
        revert Errors.InsufficientApproval();
    }
    // Transfer tokens
    IERC1155(creditToken).safeTransferFrom(from, address(this), tokenId, amount, "");
} else if (creditConfig.tokenType == CommonStructs.TokenType.ERC20) {
    // For ERC20 tokens
    if (IERC20(creditToken).allowance(from, address(this)) < amount) {
        revert Errors.InsufficientApproval();
    }
    // Transfer tokens
    IERC20(creditToken).safeTransferFrom(from, address(this), amount);
}
```

The issue is that `from` is `address(this)` (CarbonLedger), and the approval check verifies whether `CarbonLedger` has approved the target vault (`address(this)` in the context of `_depositCredits()`). The allowance will also be validated internally when performing the

transfers in the `ERC20` or `ERC1155` token. However, `CarbonLedger` never calls `approve()` or `setApprovalForAll()` on the carbon credit token to grant the target vault permission to pull tokens from `CarbonLedger`.

This causes the migration to always revert at the deposit step, making the entire migration function non functional for both ERC20 and ERC1155 token types.

Impact

The `migrateCreditBetweenClasses()` function will always revert when attempting to deposit tokens to the target vault, preventing administrators from migrating carbon credits between class vaults.

Recommendation

Before calling `depositCarbonCredits()` on the target vault, `CarbonLedger` should grant approval to the target vault. For ERC20 tokens, call `IERC20(carbonCredit).approve(targetVault, amount)`. For ERC1155 tokens, call `IERC1155(carbonCredit).setApprovalForAll(targetVault, true)`.

Status

Resolved

[M-03] CarbonClassVault#updateMaturityId - Maturity ID 0 not properly handled

Description

The `updateMaturityId()` function does not validate that `newMaturityId` is not `0`. When `newMaturityId` is `0` (which represents liquid carbon by convention), the function incorrectly adds tonnes to `maturityTokenTonnes[0]` and `maturityTonnes[0]` instead of `liquidTokenTonnes` and `liquidTokenTonnes[creditToken][tokenId]`. This breaks the accounting system and can prevent users from withdrawing liquid carbon.

Vulnerability Details

In `CarbonClassVault`, maturity ID 0 is used as a special convention to represent liquid carbon, not an actual maturity. However, the `updateMaturityId()` function does not handle the special case where `newMaturityId == 0`:

```
function updateMaturityId(
    address creditToken,
    uint256[] memory tokenIds,
    uint256 oldMaturityId,
    uint256 newMaturityId
) external onlySystemAdmin {
    // ... validation code ...
    // Verify new maturity exists and is active
    IMaturityManager.MaturityInfo memory maturityInfo =
        IMaturityManager(maturityManagerAddr).getMaturityInfo(newMaturityId);
    if (!maturityInfo.exists) {
        revert Errors.InvalidMaturityId(newMaturityId);
    }
    // For each token ID
    for (uint256 i = 0; i < tokenIds.length; i++) {
```

```
        uint256 tokenId = tokenIds[i];

        // Get current maturity tonnes for this token

                                                uint256      currentTonnes      =
maturityAccounting.maturityTokenTonnes[oldMaturityId][creditToken][tokenId];

        // Only update if there are existing tonnes

        if (currentTonnes > 0) {

            // Remove tonnes from old maturity

            maturityAccounting.maturityTokenTonnes[oldMaturityId][creditToken][tokenId] =
0;

            maturityAccounting.maturityTonnes[oldMaturityId] -= currentTonnes;

            // Add tonnes to new maturity

            maturityAccounting.maturityTokenTonnes[newMaturityId][creditToken][tokenId] =
currentTonnes;

            maturityAccounting.maturityTonnes[newMaturityId] += currentTonnes;

        }

    }

}
```

When `newMaturityId == 0`:

1. The function calls `getMaturityInfo(0)` which returns `exists: true`.
2. The function then incorrectly adds tonnes to `maturityTokenTonnes[0]` and `maturityTonnes[0]`.
3. When users attempt to withdraw liquid carbon (with `maturityId = 0`), the system checks `liquidTokenTonnes` and `liquidTokenTonnes[creditToken][tokenId]`

This breaks the accounting system and prevents proper access assets that were incorrectly migrated.

Impact

When `updateMaturityId()` is called with `newMaturityId = 0`, carbon tonnes are incorrectly tracked in maturity accounting instead of liquid accounting, breaking the accounting system and preventing users from withdrawing liquid carbon.

Recommendation

Add validation to reject `newMaturityId == 0` or handle it specially. If the intention is to allow migration to liquid carbon, add special handling: when `newMaturityId == 0`, add tonnes to `liquidTokenTonnes[creditToken][tokenId]` and `liquidTonnes` instead of `maturityTokenTonnes[0]` and `maturityTonnes[0]`. Similarly, handle the case where `oldMaturityId == 0` by reading from liquidTokenTonnes and subtracting from liquidTonnes.

Status

Resolved

[M-04] ProtocolRewardsEscrow#escrowK2Rewards - Missing Integration with ProtocolMinter

Description

The `escrowK2Rewards()` function in `ProtocolRewardsEscrow` is protected by the `onlyProtocolMinter` modifier, indicating it should be called by `ProtocolMinter`. However, `ProtocolMinter` has no function to call `escrowK2Rewards()`, creating a design implementation mismatch that prevents proper integration of the escrow workflow.

Vulnerability Details

The `escrowK2Rewards()` function in `ProtocolRewardsEscrow` uses the `onlyProtocolMinter` modifier, which checks if the caller has the `PROTOCOL_REWARDS_MINTER` role.

The function documentation explicitly states "Escrows K2 rewards from the Protocol Minter", and the modifier name `onlyProtocolMinter` further reinforces that this function is intended to be called by `ProtocolMinter`. However, in `ProtocolMinter`, there is no function that calls `escrowK2Rewards()`.

While the function can technically be called by any address that has been granted the `PROTOCOL_REWARDS_MINTER` role, the design intent suggests that `ProtocolMinter` should be the primary caller.

Impact

The escrow workflow is incomplete as `ProtocolMinter` cannot escrow K2 rewards through its intended interface, requiring manual calls or workarounds that deviate from the intended design pattern and increase operational complexity.

Recommendation

Add a function to `ProtocolMinter` that calls `escrowK2Rewards()` on the `ProtocolRewardsEscrow` contract. Alternatively, if this function is not intended to be called by `ProtocolMinter`, update the naming of the modifier and update the documentation to reflect how it should be called.

Status

Resolved

[M-05] ProtocolRewardsEscrow#transferK2ToTreasury - Incorrect access control modifier

Description

The `transferK2ToTreasury()` function in `ProtocolRewardsEscrow` uses the `onlySystemAdmin` modifier, but it is called by `K2YieldFacet.distributeK2YToTreasury()` which is part of the `RewardsManager` diamond. The function should use `onlyRewardManager` instead to allow the `RewardsManager` to execute treasury transfers as part of its reward distribution workflow.

Vulnerability Details

The `transferK2ToTreasury()` function in `ProtocolRewardsEscrow` uses the `onlySystemAdmin` modifier, however, this function is called by `K2YieldFacet.distributeK2YToTreasury()` which is part of the `RewardsManager` diamond.

```
function distributeK2YToTreasury(uint64 midNightIndex, uint256 totalK2YieldForTreasury)
    external
    onlyStateUpdater
    nonReentrant
{
    // ... validation logic ...
    // Transfer K2Y from escrow to treasury
    if (totalK2YieldForTreasury > 0) {
        RewardManagerStorage.RMConfig storage config = RewardManagerStorage.rmConfig();
IProtocolRewardsEscrow(config.protocolRewardsEscrow).transferK2ToTreasury(totalK2YieldFor
Treasury);
    }
}
```

Impact

The `RewardsManager` cannot execute treasury transfers as part of its reward distribution workflow.

Recommendation

Change the `transferK2ToTreasury()` function to use the `onlyRewardManager` modifier instead of `onlySystemAdmin`.

Status

Resolved

[M-06] ProtocolMinter#transferK2ToRewardManager - Calls IRewardsEscrow non existent function

Description

The `transferK2ToRewardManager()` function in `ProtocolMinter` attempts to call `transferToRewardManagers()` on the `ProtocolRewardsEscrow` contract, but this function does not exist. The `ProtocolRewardsEscrow` contract implements `transferK2ToRewardManager()` instead, which can be called directly by the `RewardManager` contract, making the `ProtocolMinter` wrapper function both broken and redundant.

Vulnerability Details

The `transferK2ToRewardManager()` function in `ProtocolMinter` calls a non existent function:

```
function transferK2ToRewardManager(uint256 amount)
    external
    nonReentrant
    onlyRewardManager
    nonZeroAmount(amount)
    whenNotPaused
{
    // Call the RewardsEscrow contract to transfer K2 to RewardManager
    IRewardsEscrow(_config.rewardsEscrow).transferToRewardManagers(amount);
    emit K2TransferredToRewardManager(amount);
}
```

The function attempts to call `IRewardsEscrow.transferToRewardManagers()` on the `ProtocolRewardsEscrow` contract. However, `ProtocolRewardsEscrow` implements `IProtocolRewardsEscrow`, not `IRewardsEscrow`. The `IProtocolRewardsEscrow` interface defines `transferK2ToRewardManager()` function, not `transferToRewardManagers()`.

The actual implementation in `ProtocolRewardsEscrow` is:

```
function transferK2ToRewardManager(uint256 totalK2YieldForRewardManager)

    external

    onlyRewardManager

    whenNotPaused

    nonReentrant

{

    if (escrowBalance.k2RewardsEscrowed < totalK2YieldForRewardManager) {

            revert  Errors.InsufficientK2RewardsEscrowed(escrowBalance.k2RewardsEscrowed,
totalK2YieldForRewardManager);

    }

    escrowBalance.k2RewardsEscrowed -= totalK2YieldForRewardManager;

                                IERC20(config.k2Token).safeTransfer(config.rewardManager,
totalK2YieldForRewardManager);

    emit K2RewardsTransferredToRewardManager(totalK2YieldForRewardManager);

}
```

This function is protected by the `onlyRewardManager` modifier, which means the `RewardManager` contract can call it directly without going through `ProtocolMinter`. The `ProtocolMinter.transferK2ToRewardManager()` function is therefore both broken (will revert due to non-existent function) and redundant (RewardManager can call the escrow directly).

Impact

The `transferK2ToRewardManager()` function in `ProtocolMinter` will always revert when called, preventing the intended K2 transfer workflow through `ProtocolMinter`.

Recommendation

Either remove the `transferK2ToRewardManager()` function from `ProtocolMinter` if `RewardManager` is intended to call the escrow directly, or fix the function to call the correct function name `transferK2ToRewardManager()` on `IProtocolRewardsEscrow` instead of `transferToRewardManagers()` on `IRewardsEscrow`.

Status

Resolved

[M-07] KvcmLPStakingFacet#stakeLPWithPermit, stakeLPForWithPermit - Permit functions fail due to incorrect msg.sender in token transfer

Description

The `stakeLPWithPermit()` and `stakeLPForWithPermit()` functions use `this.stakeLP()` and `this.stakeLPFor()` respectively, which causes `msg.sender` to be the contract address instead of the original user. This results in failed token transfers since the contract attempts to transfer tokens from itself, but the permit was executed by the original user.

Vulnerability Details

In `stakeLPWithPermit()`, after executing the permit for the original user:

```
function transferK2ToRewardManager(uint256 totalK2YieldForRewardManager)

    external

    onlyRewardManager

    whenNotPaused

    nonReentrant

{

    if (escrowBalance.k2RewardsEscrowed < totalK2YieldForRewardManager) {

            revert  Errors.InsufficientK2RewardsEscrowed(escrowBalance.k2RewardsEscrowed,
totalK2YieldForRewardManager);

    }

    escrowBalance.k2RewardsEscrowed -= totalK2YieldForRewardManager;

                              IERC20(config.k2Token).safeTransfer(config.rewardManager,
totalK2YieldForRewardManager);

    emit K2RewardsTransferredToRewardManager(totalK2YieldForRewardManager);

}
```

The function calls `this.stakeLP()`, which is an external call. When making an external call using `this.functionName()`, `msg.sender` inside the called function becomes the contract's own address instead of the original caller.

The `stakeLP()` function then delegates to a pair specific facet via `delegatecall`. The delegated facet's `stakeLP()` function, inside `_stakeLP()`, attempts to transfer tokens from `msg.sender`.

Since `msg.sender` is the contract itself, the transfer attempts to pull tokens from the contract's own balance. However, the permit was executed by the original user, granting approval to the contract, but the tokens remain in the original user's account. The contract does not have these tokens, causing `safeTransferFrom()` to revert.

The same issue exists in `stakeLPForWithPermit()`, which calls `this.stakeLPFor()`.

Impact

Both permit functions will always revert when attempting to transfer tokens, preventing users from staking LP tokens using permit functionality.

Recommendation

Change `stakeLP()` and `stakeLPFor()` from `external` to `public` visibility, and update `stakeLPWithPermit()` and `stakeLPForWithPermit()` to call them directly without using `this.functionName()`.

Alternatively, consider using `delegatecall` as done in the other methods to the `stakeLPWithPermit()` and `stakeLPForWithPermit()` functions of the registered module for the lp pair.

Status

Resolved

[M-08] KvcmStakingFacet#reallocateKvcm - Incorrect removal of maturity aggregate allocation

Description

The `reallocateKvcm()` function incorrectly removes the maturity aggregate entry from `priceAllocationsByMaturity` when a single lock fully reallocates, breaking deallocation functionality for other locks sharing the same maturity and carbon class combination.

Vulnerability Details

The `reallocateKvcm()` function manages two types of allocation tracking structures:

- `lockAllocations[lockId][carbonClass]`: Tracks allocations for a specific lock
- `priceAllocationsByMaturity[maturityId][carbonClass]`: Tracks aggregate allocations across all locks with the same maturity

When updating allocations for the `fromCarbonClass`, the function checks if `existingLockAllocation == amount` to determine whether to remove entries.

However, it incorrectly removes the maturity aggregate entry based on this lock specific check:

```
if (existingLockAllocation == amount) {
    sKvcm.lockAllocations[lockId].remove(fromCarbonClass);
    sKvcm.priceAllocationsByMaturity[lockInfo.maturityId].remove(fromCarbonClass);
} else {
    sKvcm.lockAllocations[lockId].set(fromCarbonClass, existingLockAllocation - amount);
    sKvcm.priceAllocationsByMaturity[lockInfo.maturityId].set(
        fromCarbonClass, existingMaturityAllocation - amount
    );
}
```

The maturity entry should only be removed when `existingMaturityAllocation == amount`, not when a single lock's allocation equals the amount. This is correctly implemented in `deallocateKvcm()`, which checks the aggregate value before removal.

When multiple locks share the same maturity and carbon class:

1. Lock1 has 100 allocated to ClassA, Lock2 (same maturity) also has 100 allocated to ClassA, `priceAllocationsByMaturity[maturityId][ClassA] = 200`
2. User reallocates Lock1's 100 from ClassA to ClassB
3. Code removes `priceAllocationsByMaturity[maturityId][ClassA]` because `existingLockAllocation == amount`
4. Lock2 still has 100 allocated to ClassA, but the maturity entry is gone
5. When Lock2 attempts to deallocate, `deallocateKvcm()` fails because it expects the maturity entry to exist

Impact

Users with locks sharing the same maturity and carbon class cannot deallocate their allocations after another lock fully reallocates from that class, resulting in locked funds and denial of service.

Recommendation

Modify the condition to check the aggregate maturity allocation value instead of the lock specific value before removing the maturity entry:

```
if (existingLockAllocation == amount) {
    sKvcm.lockAllocations[lockId].remove(fromCarbonClass);
} else {
    sKvcm.lockAllocations[lockId].set(fromCarbonClass, existingLockAllocation - amount);
}
if (existingMaturityAllocation == amount) {
    sKvcm.priceAllocationsByMaturity[lockInfo.maturityId].remove(fromCarbonClass);
} else {
    sKvcm.priceAllocationsByMaturity[lockInfo.maturityId].set(
```

```
        fromCarbonClass, existingMaturityAllocation - amount

    );

}
```

Status

Resolved

[M-09] StakingManagerPauseFacet#onlySystemAdmin - Pause Controls Require Owner and Both Roles

Description

The pause and unpause functions in `StakingManagerPauseFacet` are gated by `onlySystemAdmin`, but the modifier logic requires the caller to be the diamond owner and also hold both `CARBON_SYSTEM_ADMIN` and `KVCM_STAKING_MANAGER_ADMIN`. This may make emergency pause controls unusable in production unless multiple privileges are concentrated in one address.

Vulnerability Details

`onlySystemAdmin` reverts if any of the three checks fails, due to `||`:

```
// in onlySystemAdmin()

if (msg.sender != LibDiamond.contractOwner() || !isSystemAdmin || !isStakingManagerAdmin)
{

        revert Errors.NotAuthorized();

    }
```

All pause/unpause functions are protected by this modifier.

Impact

Pause/unpause functionality may be operationally unreachable unless the same address is simultaneously: The diamond owner, and granted `CARBON_SYSTEM_ADMIN`, and Granted `KVCM_STAKING_MANAGER_ADMIN`.

Recommendation

Replace the condition with the intended policy, e.g:

Owner OR (SystemAdmin AND StakingManagerAdmin), or Owner OR SystemAdmin, depending on governance requirements.

Status

Resolved

[M-10] MaturityManager - Contract inherits Pausable but exposes no pause/unpause functions

Description

The `MaturityManager` contract inherits from `PausableUpgradeable` and uses the `whenNotPaused` modifier, but it fails to expose external `pause()` or `unpause()` functions, making the pause functionality inaccessible.

Vulnerability Details

The contract initializes `PausableUpgradeable` in the initialize function and applies the `whenNotPaused` modifier to `updateMaxMaturityId()`. However, there are no external functions implemented to call the internal `_pause()` or `_unpause()` methods.

```
// MaturityManager.sol

contract MaturityManager is IMaturityManager, UUPSUpgradeable, PausableUpgradeable {

    // ...

    function initialize(...) public initializer {

        __Pausable_init();

        // ...

    }

    function updateMaxMaturityId(...) external onlyCarbonMaturityManager whenNotPaused {

        // ...

    }

    // No pause() or unpause() functions defined

}
```

Impact

The pause mechanism is completely dysfunctional. In an emergency situation, the `updateMaxMaturityId` function cannot be paused, preventing administrators from stopping potentially malicious or erroneous updates to maturity parameters.

Recommendation

Implement `pause()` and `unpause()` functions restricted to the System Admin or an appropriate role.

Status

Resolved

[M-11] MaturityManager#updateMaxMaturityId - Timestamp corruption due to uninitialized mapping

Description

The `updateMaxMaturityId()` function uses an uninitialized mapping value (i.e default `0`) as the base timestamp when extending beyond initialized IDs, causing all new maturities to have incorrect, ancient timestamps starting from 1970 instead of continuing from the proper future date.

Vulnerability Details

In the `updateMaxMaturityId()` function, it fetches `latestTimestamp` directly from `maturityTimestamps[currentMaxMaturityId]`, but for IDs like 800 (set as max during init but not stored in the mapping for IDs 41-800), this returns `0`. The loop then adds `MATURITY_PERIOD` to `0`, storing wrong timestamps (e.g., 7776000 for ID 801, which is March 1970). This corrupts the chain for all subsequent extensions.

Impact

This corrupts all new maturity timestamps after the first extension beyond initialized IDs, breaking time-based logic in dependent contracts like staking and rewards, potentially causing incorrect lock periods, failed validations, or financial losses in the protocol.

Recommendation

Use `_timestampFor(currentMaxMaturityId)` instead of `maturityTimestamps` mapping to fetch value.

Status

Resolved

[M-12] KvcmIncentivesCurveFacet#_getLockClaimValueAtMidnightIndex - underestimates yield when unprocessed deposits exist

Description

`_getLockClaimValueAtMidnightIndex()` computes the lock value using shares amount that excludes unprocessed deposits, but subtracts a principal value that includes those deposits, causing under calculation of yield in certain states.

Vulnerability Details

The function first computes effective principal as `effShares * pps`, where `effShares` comes from `_previewSharesWithAvailablePps()` and ignores any deposits whose `ppsAtRoll` is still zero, but then subtracts `lock.rawPrincipal`, which is incremented for every deposit regardless of whether it has been processed into shares. This creates a mismatch when there are still unprocessedDeposits in the lock:

```
uint256 ppsAtMidnightIndex =

                KvcmIncentivesCurveMathLib.getPricePerShareAtMidnightIndex(midnightIndex,
lock.maturityId);

uint256 effShares =

    _previewSharesWithAvailablePps(lock, RewardManagerStorage.kvcmLockYieldLayout());

uint256  effectivePrincipalAtMidnightIndex  =  Math.mulDiv(effShares,  ppsAtMidnightIndex,
Constants.WAD);

claimValue = effectivePrincipalAtMidnightIndex > lock.rawPrincipal

    ? effectivePrincipalAtMidnightIndex - lock.rawPrincipal

    : 0;
```

In this situation, some principal is counted in `rawPrincipal` but not in `effShares`, so those recent deposits reduce the computed yield instead of being neutral.

Impact

Here when a lock has unprocessed deposits (e.g. during partial claims or intermediate midnight indices), the reported or paid yield for that lock is systematically too low

because fresh principal is included in `lock.rawPrincipal` but excluded from the `effShares` used to value the position, leading to user underpayment and incorrect yield accounting.

Recommendation

Update the logic to exclude the principal from the unprocessed deposits before the subtraction.

Status

Resolved

[M-13] KvcmIncentivesCurveFacet - Partial claimed Kvcm locks remain in global totals, skewing yield curve

Description

When a Kvcm lock is partially claimed in `KvcmIncentivesCurveFacet`, its principal and shares are not removed from the global maturity totals, so "dead" locks continue to count in `totalRawPrincipal` and `totalSharesForMaturity` even though the user's principal is already unlocked.

Vulnerability Details

On lock updates, both the per-lock and global maturity totals are incremented:

```
lock.rawPrincipal += deltaAmount;

yieldLayout.currentMaturityTotals[maturityId].totalRawPrincipal += deltaAmount;
```

For a full unlock, `handleUnlockAndClaim()` eventually calls `_handleUnlockAndBurn()`, which decrements both totals and zeroes the lock:

```
yieldLayout.currentMaturityTotals[lock.maturityId].totalSharesForMaturity -= lock.shares;

// burn lock shares

yieldLayout.currentMaturityTotals[lock.maturityId].totalRawPrincipal -=
lock.rawPrincipal;

// update maturity totals

lock.rawPrincipal = 0;

lock.shares = 0;
```

However, in the partial-claim path (`_handlePartialClaim()`), the function only updates `yieldDistributed` and status to `PARTIAL_CLAIMED` and never calls `_handleUnlockAndBurn()` or adjusts `totalRawPrincipal` / `totalSharesForMaturity`. At the same time, `KvcmStakingFacet._unlockKvcm()` fully unlocks the user's principal and burns the Kvcm lock NFT, so these partially-claimed locks remain counted in global totals even though they are no longer real locked positions.

Impact

The yield-curve and discount-factor logic that relies on `currentMaturityTotals[maturityId].totalRawPrincipal` and `totalSharesForMaturity` will overestimate how much Kvcm is locked at a maturity, which distorts curve shape, zero-coupon yields, and subsequent yield distributions. This leads to mispriced incentives and unfair distribution of future rewards across all remaining participants, even though no principal is directly stolen.

Recommendation

When performing a partial claim, adjust the global maturity totals to remove the portion of the lock that has effectively been unlocked, or split the lock into an active remainder and a settled portion such that only the truly active part continues to contribute to totalRawPrincipal and totalSharesForMaturity, and ensure _handleUnlockAndBurn (or equivalent logic) is invoked once a lock is fully exited.

Status

Resolved

[M-14] KvcmIncentivesCurveFacet#_validateLockingInterval - function ignores yieldDistributionInterval and only checks active range

Description

The `_validateLockingInterval()` function reads `yieldDistributionInterval` from storage but never uses it, and only checks that maturityId is within the active maturity range, contrary to its comment about enforcing a minimum yield period.

Vulnerability Details

The function currently looks like this:

```
function _validateLockingInterval(uint256 maturityId) internal view {

                                        uint256     yieldDistributionInterval     =
RewardManagerStorage.kvcmLockYieldLayout().yieldDistributionInterval;

        (uint256 start, uint256 end) =


IMaturityManager(RewardManagerStorage.configLayout().maturityManager).getActiveMaturityRa
nge();

        if (start > maturityId || end < maturityId) {

            revert Errors.InvalidLockingInterval(maturityId);

        }

    }
```

The comment above `_validateLockingInterval()` and the call site ("revert if less than one yield period until maturity") suggest it should use `yieldDistributionInterval` to enforce that locks cannot be created too close to maturity, but the variable is never checked and the only enforced condition is that the maturity is currently active (which is already validated on the staking side).

Impact

The protocol does not enforce the intended "at least one yield interval before maturity" constraint, allowing users to create or update locks closer to maturity than the design

appears to intend, which can slightly change the economic behavior of the incentives model but does not directly compromise funds or core safety.

Recommendation

Either remove the unused `yieldDistributionInterval` read and update comments to match the actual behavior, or properly enforce the intended constraint by checking that the time between now and the maturity timestamp is at least one `yieldDistributionInterval` and reverting otherwise.

Status

Resolved

[M-15] K2YieldCurveFacet#updateSchedule - resets cumulative token allocation breaking emission continuity

Description

The `updateSchedule()` function resets the curve's baseline (P0) to the current progress level without offsetting the cumulative token calculation. This causes `cumulativeAllocatedAt` to drop significantly (potentially to zero) immediately after an update, violating the monotonic increase required for correct token emission.

Vulnerability Details

The function `updateSchedule()` is designed to re-anchor the yield curve. It calculates the current normalized progress `fNow` and sets the new start parameter `P0_wad` to this value.

```
// K2YieldCurveFacet.sol

function updateSchedule(...) external {

    uint256 fNow = _pNormAt(newStart); // Current progress

    s.start = newStart;

    s.P0_wad = fNow; // New baseline is set to current progress

    // ...

}
```

The `cumulativeAllocatedAt` function calculates total tokens as `_pNormAt(t) * rewardCap`. The helper `_pNormAt()` returns 0 when `P(t) == P0`.

```
function _pNormAt(uint256 ts) internal view returns (uint256) {

    uint256 p = _pRawAt(ts);

    if (p <= s.P0_wad) return 0; // Returns 0 at the new start

    // ...

}
```

By resetting the curve such that P(newStart) == P0, the normalized progress at newStart becomes 0. Consequently, `cumulativeAllocatedAt(newStart)` resets to 0, "forgetting" all previously allocated tokens.

Impact

This discontinuity disrupts the token emission schedule. Since daily emissions are derived from the difference in cumulative allocations over time, a reset to zero can lead to incorrect emission calculations, halted rewards, or accounting errors where the system believes no tokens have been emitted yet.

Recommendation

Introduce a state variable to track the cumulative allocated amount at the time of an update. Add this offset to the result of `cumulativeAllocatedAt` to ensure the value remains monotonically increasing across updates.

Status

Resolved

[M-16] K2YieldCurveFacet#pullK2YieldFromEscrow - dailyPulled mapping collision after updateSchedule resets start time

Description

The `pullK2YieldFromEscrow()` function uses a `midnightIndex` calculated relative to `s.start`. When `updateSchedule()` changes `s.start`, the midnightIndex calculation resets, causing new days to map to low indices (1, 2, 3...) that may already contain data in the dailyPulled mapping from the previous schedule. This collision prevents pulling rewards for new days if the old data indicates the limit is already reached.

Vulnerability Details

The midnightIndex is derived relative to the current s.start:

```
uint256 currentMidnightIndex = ((block.timestamp - s.start) / 1 days) + 1;
```

When rewards are pulled, `s.dailyPulled[midnightIndex]` is updated.

If `updateSchedule()` is called, `s.start` is updated to a new timestamp (usually block.timestamp or slightly in the future).

After this update, `currentMidnightIndex` for the new schedule starts at 1 again.

However, `s.dailyPulled` is a persistent mapping in storage and is not cleared.

```
mapping(uint256 => uint256) dailyPulled; // in Storage
```

If dailyPulled[1] was used in the previous schedule, the new schedule's Day 1 will read that non-zero value.

If `todaysPulled + amount > emissionLimit` check fails because todaysPulled includes old data, no new rewards can be pulled for that day.

Impact

Reward pulling will be blocked for the initial days of a new schedule equal to the number of days rewards were pulled in the previous schedule. This effectively locks funds in the escrow and disrupts the reward distribution mechanism.

Recommendation

Consider introducing a mechanism to prevent this collision by either reading from different positions based on the `s.start` or cleaning the storage on its new first usage after a restart.

Status

Resolved

[M-17] RiskyYieldUtilsLib#_settleYieldForLP - Underflow when processing index 0 entries

Description

The `_settleYieldForLP()` function in `RiskyYieldUtilsLib` causes a revert due to arithmetic underflow when processing an LP position entry where `settledMidnightIndex` is 0. This occurs because the code attempts to subtract 1 from the index without checking if it is already 0.

Vulnerability Details

When settling yield for an LP position, the code checks if `positionEntry.settledMidnightIndex` equals currentIdx (the index being processed). If they match, it attempts to fetch the accumulator from the previous index (`settledMidnightIndex - 1`).

```
// RiskyYieldUtilsLib.sol

if (positionEntry.settledMidnightIndex == currentIdx) {

    // Underflows if positionEntry.settledMidnightIndex == 0

                                    accPosLastSettled                                    =
r.lpRYieldAccSnapshot[lpToken][maturityId][positionEntry.settledMidnightIndex - 1]

        .midnightAccumulator;

}
```

Since MaturityManager can return a midnight index of 0 (e.g., at the start of the protocol or before the first maturity), valid entries can be created with index 0. When such an entry is processed, 0 - 1 results in an arithmetic underflow, causing the transaction to revert and potentially locking user rewards or preventing settlement.

Impact

Users who deposited at the very start (midnight index 0) will be unable to settle or claim their LP rewards because any transaction attempting to process that entry will revert. This effectively locks their accrued rewards.

Recommendation

Add a check to handle the case where settledMidnightIndex is 0. If it is 0, accPosLastSettled should likely be 0 (assuming the accumulator starts at 0 before index 0).

```
if (positionEntry.settledMidnightIndex == currentIdx) {

    if (positionEntry.settledMidnightIndex == 0) {

        accPosLastSettled = 0; // or initial accumulator value

    } else {

                                                                accPosLastSettled       =
r.lpRYieldAccSnapshot[lpToken][maturityId][positionEntry.settledMidnightIndex - 1]

            .midnightAccumulator;

    }

}
```

Status

Resolved

[M-18] RiskyYieldFacet#computeRY - reverts because IntervalData.kVCMYield is never set

Description

The `computeRY()` function strictly requires `intervalData.kVCMYield` to be non-zero to proceed with risky yield computation. However, a code analysis reveals that no function in the codebase sets this `kVCMYield` field in the IntervalData struct, leading to a permanent revert in `computeRY()`.

Vulnerability Details

The function `computeRY()` retrieves `intervalData` for a given midnight index and checks `kVCMYield`:

```
// RiskyYieldFacet.sol

RewardManagerStorage.IntervalData storage intervalData = kvcmLockYieldLayout.intervalData[midnightIndex];

// ...

uint256 kVCMYield = intervalData.kVCMYield;

if (kVCMYield == 0) {

    revert Errors.KVCMYieldNotDistributed(midnightIndex);

}
```

Searching the codebase (and based on the provided context), there is no assignment to `intervalData.kVCMYield`. As a result, `kVCMYield` remains `0` by default, and `computeRY` will always revert with `KVCMYieldNotDistributed`, breaking the risky yield distribution mechanism.

Impact

The risky yield computation and distribution system is completely non-functional. Since `computeRY` is a critical step in determining rewards, no risky yield rewards can ever be calculated or distributed.

Recommendation

Implement the logic to populate `kVCMYield` in IntervalData. This should likely happen in a function responsible for settling or recording the daily kVCM yield.

Status

Resolved

[M-19] CarbonVaultFactory - Vaults deployed with version 0 cannot be upgraded

Description

The `registerImplementation()` function allows registering an implementation with version 0. However, `upgradeVault()` reverts if the current version of a vault is 0, making any vault deployed with version 0 permanently non-upgradeable.

Vulnerability Details

The `registerImplementation()` function does not validate that the version parameter is non-zero, allowing an admin to register version 0.

```
// CarbonVaultFactory.sol

function registerImplementation(uint256 version, address implementation) external onlySystemAdmin notPaused {

    // ... no check for version > 0

    _implementations[version] = implementation;

}
```

When `deployVault()` is called with implementationVersion as `0`, the vault is deployed successfully and `_vaultInfo[vault].currentVersion` is set to `0`.

```
function deployVault(..., uint256 implementationVersion) ... {

    // ...

    _vaultInfo[vault] = VaultInfo({currentVersion: implementationVersion, ...});

}
```

However, the `upgradeVault()` function includes a check that reverts if `currentVersion` is 0, treating it as an untracked vault.

```
function upgradeVault(address vault, uint256 targetVersion) external ... {

    if (_vaultInfo[vault].currentVersion == 0) revert Errors.ClassVaultNotTracked();

    // ...
```

```
}
```

This effectively locks any vault deployed with version 0 to that specific implementation, preventing future upgrades.

Impact

Vaults deployed using implementation version 0 cannot be upgraded, breaking the intended upgradeability mechanism for those specific instances. This could prevent critical fixes or improvements from being applied to early deployments.

Recommendation

Add a check in registerImplementation to ensure version is greater than 0.

Status

Resolved

[M-20] CarbonClassVault#getAvailableTonnes - Coupons can become unbacked by liquid tonnes after withdrawals

Description

`getAvailableTonnes()` uses `totalTonnes - reservedTonnes` (which includes forward tonnes) to gate withdrawals, while coupon minting requires `liquidTonnes >= reservedTonnes + tonnes`. This mismatch allows liquid withdrawals that deplete the liquid backing for coupons, breaking the invariant that coupons should be backed by liquid tonnes.

Vulnerability Details

When issuing coupons, `_issueCoupons()` enforces that coupons are backed by liquid tonnes:

```
function _issueCoupons(address account, uint256 tonnes) internal {

                                            uint256           liquidTonnes           =
CarbonVaultStorage.getMaturityAccountingStorage().liquidTonnes;

    uint256 reservedTonnes = CarbonVaultStorage.getCouponStorage().reservedTonnes;

                  if     (liquidTonnes     <     reservedTonnes     +     tonnes)     revert
Errors.InsufficientAvailableTonnes();

    CarbonVaultStorage.getCouponStorage().reservedTonnes += tonnes;

    _mint(account, tonnes);

}
```

However, when withdrawing carbon credits, the availability check uses `getAvailableTonnes()`, which calculates available tonnes as `totalTonnes - reservedTonnes`:

```
function getAvailableTonnes() public view returns (uint256) {

    uint256 totalTonnes = CarbonVaultStorage.getVaultAccountingStorage().totalTonnes;

    uint256 reservedTonnes = CarbonVaultStorage.getCouponStorage().reservedTonnes;

    return totalTonnes >= reservedTonnes ? totalTonnes - reservedTonnes : 0;
```

```
}
```

The withdrawal function uses this check before enforcing per bucket liquidity:

```
function _withdrawCarbonCredits(...) internal {

            uint256   tonnesWithdrawn   =   CarbonTonnageLib.tokenAmountToTonnes(amount,
config.conversion);

    // Check if there are enough available (unreserved) tonnes

    if (!hasEnoughAvailableTonnes(tonnesWithdrawn)) {

        revert Errors.CannotWithdrawReservedTonnes();

    }

    // ... later checks liquid tonnes ...

    if (maturityId == 0) {

                    if   (maturityAccounting.liquidTonnes   <   tonnesWithdrawn)   revert
Errors.InsufficientLiquidTonnes();

        maturityAccounting.liquidTonnes -= tonnesWithdrawn;

    }

    vaultAccounting.totalTonnes -= tonnesWithdrawn;

}
```

Since `totalTonnes` includes both liquid and forward tonnes, an admin or liquidity manager can:

1. Deposit large amounts of forward tonnes (increasing `totalTonnes` but not `liquidTonnes`)
2. Issue coupons backed by available liquid tonnes (increasing `reservedTonnes`)
3. Withdraw liquid tonnes using `withdrawCarbonCredits()` with `maturityId = 0`

The withdrawal passes the `hasEnoughAvailableTonnes()` check because `totalTonnes` (including forward tonnes) is large enough, but it decreases `liquidTonnes` while leaving `reservedTonnes` unchanged. This can result in a state where `reservedTonnes > liquidTonnes`, breaking the invariant that coupons should be backed by liquid tonnes.

Impact

Coupons can become partially or fully unbacked by liquid tonnes, breaking the invariant that `reservedTonnes <= liquidTonnes`. While coupons continue to function for their intended discount mechanism in `OperationsFacet.retireCarbonCreditViaAggregator()` and `OperationsFacet.swapCarbonCreditForKvcm()`, new coupon issuance will be blocked until liquid tonnes are replenished through forward maturity or new deposits.

Recommendation

Update the withdrawal check in `_withdrawCarbonCredits()` to enforce `liquidTonnes >= reservedTonnes + tonnesWithdrawn` when withdrawing liquid carbon (`maturityId = 0`), ensuring that liquid withdrawals cannot deplete the liquid backing for coupons.

Status

Resolved

[M-21] OperationsMath#_computeExponentialGrowth - rateDelta is computed before timeDelta is capped and rateDelta is never recomputed

Description

`OperationsMath._computeExponentialGrowth()` is used by the AAM rate limiter to compute effective KVCM/K2 allocation limits for a carbon class, which are then used in swap quote pricing. The function attempts to cap the growth window using `MAX_TIME_DELTA = 30 days`, but `rateDelta` is computed before `timeDelta` is capped and is never recomputed. As a result, the intended `MAX_TIME_DELTA` protection is effectively bypassed and the limiter can become much looser than intended after long inactivity.

Vulnerability Details

In `_computeExponentialGrowth`, `rateDelta` is computed from the uncapped `timeDelta`, then `timeDelta` is capped, but the cap does not affect `rateDelta`:

```
function _computeExponentialGrowth(

    uint256 snapshotValue,

    uint256 snapshotTime,

    uint64 rate,

    uint256 currentTimestamp

) internal pure returns (uint256 limit) {

    validateRateLimitParamSet(rate);

        uint256 timeDelta = currentTimestamp > snapshotTime ? (currentTimestamp - snapshotTime) : 0;

    uint256 rateDelta = uint256(rate) * timeDelta;



    uint256 MAX_TIME_DELTA = 30 days;

    if (timeDelta > MAX_TIME_DELTA) {

        timeDelta = MAX_TIME_DELTA;

    }
```

```
    if (rateDelta > Constants.UD_EXP_MAX_INPUT) {
        rateDelta = Constants.UD_EXP_MAX_INPUT;
    }

    UD60x18 growth = ud(rateDelta).exp();
    // ...
    limit = unwrap(ud(snapshotValue).mul(growth));
}
```

This function is used by the AAM rate limiter to compute the effective KVCM/K2 allocation limits for a carbon class. `RateLimiterLib.calculateCurrentEffectiveKvcmAllocation()` and `RateLimiterLib.calculateCurrentEffectiveK2Allocation()` call `_computeExponentialGrowth()`, and `RateLimiterLib.calculateAllocationValuesForSwapQuote()` applies `min(actualStake, effectiveAllocation)`. `QuoterRouterFacet._getNormalizedClassAllocationsForQuote()` then uses these effective values for `QuoteType.SWAP` and normalizes them into the pricing inputs, and the resulting quote is consumed by `OperationsFacet.swapCarbonCreditForKvcm()` to mint and transfer KVCM based on the computed `kvcmOut`.

After sufficiently long inactivity, `rateDelta` reaches the clamp `Constants.UD_EXP_MAX_INPUT`, making the effective allocation much larger than intended. With the default KVCM rate, this happens after roughly ~6 months of inactivity since the last snapshot update (because `rateDelta = rate * timeDelta`).

Impact

The intended `MAX_TIME_DELTA` safety bound is not enforced for the value actually used (`rateDelta`). After long inactivity (e.g., on stale/low-activity carbon classes), the effective allocation limit used in swap pricing can become much larger than intended,

which can materially change swap quotes and the amount of KVCM minted in `swapCarbonCreditForKvcm()`.

Recommendation

Cap `timeDelta` before computing `rateDelta`, or recompute `rateDelta` after applying the cap. This ensures the intended "max inactivity window" protection is actually applied to the exponential growth term used by the rate limiter.

Status

Resolved

[M-22] K2StakingFacet, KvcmStakingFacet - Incorrect lambda checkpoint timing when beta changes

Description

In `K2StakingFacet`, `updateRewardsProportions()` is called after beta updates and with post-change `totalK2Allocations`, causing the elapsed time interval to be accounted using post-change parameters instead of pre-change ones. In `KvcmStakingFacet`, beta is updated but `updateRewardsProportions()` is never called, causing lambda checkpoints to be missed when beta changes.

Vulnerability Details

The `updateRewardsProportions()` function in `RiskyYieldFacet` accumulates time-weighted lambda values by calculating the increment for the elapsed time since the last checkpoint:

```
function updateRewardsProportions(uint256 midNightIndex, uint256 totalK2Allocations)
    public
    onlyStateUpdater
    whenNotPaused
{
    // ...
    uint256 betaSq = _getBetaSqForDay(midNightIndex);
    uint256 beta = Math.sqrt(betaSq);
    // Calculate instantaneous lambdas using totalK2Allocations parameter
    uint256 lambdaGG = _computeLambdaGG(
        LambdaGGParams({
            totalK2Allocations: totalK2Allocations,
            // ...
        })
    );
```

```
    // Calculate deltaTime as ELAPSED time since last update or midnight start
    uint256 deltaTime;
    if (lastUpdateTimestamp > 0) {
        deltaTime = block.timestamp - lastUpdateTimestamp;
    } else {
        deltaTime = block.timestamp - midnightStartTimestamp;
    }
    // Accumulate time-weighted lambdas for the elapsed period
    dailyLambdaCache.tblambdaGG += _calculateTimeBetalambda(deltaTime, beta, lambdaGG);
    dailyLambdaCache.tblambdaG += _calculateTimeBetalambda(deltaTime, beta, lambdaG);
    dailyLambdaCache.tblambdaQ += _calculateTimeBetalambda(deltaTime, beta, lambdaQ);
}
```

In `_allocateK2()`, `_deallocateK2()`, and `_reallocateK2()`, the code updates beta first and then calls `updateRewardsProportions()` with the new `totalK2Allocations`:

```
function _allocateK2(uint256 newAllocationAmount, address carbonClass) internal {
    // ...
    uint256 newTotalK2Allocations = l.totalK2Allocations + newAllocationAmount;
    // Update storage
    l.totalK2Allocations = newTotalK2Allocations;
    // Update beta FIRST (this changes the beta value)
    IRiskyYieldFacet(config.rewardManager).updateBetaSqForK2Allocation(
        newAllocationAmount, carbonClass, currentDayId
    );
    // Then checkpoint with NEW total and NEW beta
            IRiskyYieldFacet(config.rewardManager).updateRewardsProportions(currentDayId,
newTotalK2Allocations);
```

```
}
```

The issue is that `updateRewardsProportions()` calculates the lambda increment for the elapsed time period (since the last checkpoint) using:

- The current beta (which was just updated) instead of the previous beta
- The new `totalK2Allocations` instead of the previous `totalK2Allocations`

This means the elapsed time interval is incorrectly accounted using post-change parameters. The correct approach is to checkpoint the elapsed period using the pre-change parameters, then update beta for future calculations.

In `allocateKvcm()`, `deallocateKvcm()`, and `reallocateKvcm()`, beta is updated but `updateRewardsProportions()` is never called:

```
function allocateKvcm(uint256 lockId, address carbonClass, uint256 amount) external {
    // ...
    // Update beta but never checkpoint lambdas
    IRiskyYieldFacet(sConfig.rewardManager).updateBetaSqForKvcmAllocation(
        amount,
        carbonClass,

SafeCast.toUint64(IMaturityManager(sConfig.maturityManager).getMidnightIndexForTimestamp(
block.timestamp))
    );
    // No call to updateRewardsProportions()
}
```

When beta changes due to Kvcm allocations/deallocations, the lambda checkpoints are not accumulated. This means the time-weighted lambdas are not properly tracked during the day, especially if multiple allocations and deallocations happen. The elapsed time since the last checkpoint (or since midnight) is never accounted for when beta changes.

Impact

Incorrect reward distribution occurs because time-weighted lambda accumulations use wrong values.

Recommendation

For `K2StakingFacet`: Call `updateRewardsProportions()` before updating beta, and pass the previous `l.totalK2Allocations` value instead of the new one. This ensures the elapsed time period is accounted using pre-change parameters.

For `KvcmStakingFacet`: Call `updateRewardsProportions()` before updating beta in `allocateKvcm()`, `deallocateKvcm()`, and `reallocateKvcm()`. Pass the current `totalK2Allocations` from the staking layout to ensure lambda checkpoints are accumulated when beta changes.

Status

Resolved

Low

[L-01] CarbonLedger.sol#unregisterCreditForClass - ERC1155 partial unregister locks remaining tokenIds

Description

Unregistering a subset of ERC1155 tokenIds marks the entire credit/class as unregistered, blocking further operations on remaining tokenIds until re-registration.

Vulnerability Details

`registerCreditForClass()` sets `creditStatusForClass[classVault][carbonCredit]` to `Registered`, but ERC1155 registrations track tokenIds only in the vault. In `unregisterCreditForClass()`, after removing the provided tokenIds, the contract sets `creditStatusForClass[classVault][carbonCredit] = Status.Unregistered` regardless of other registered tokenIds.

`_isCreditRegisteredForClass()` returns true only when that status is `Registered` and is enforced at the start of `unregisterCreditForClass()`, `migrateCreditBetweenClasses()`, and `_validateLiquidityOperation()` (used by `addCreditLiquidityToClassVault()` and `removeCreditLiquidityFromClassVault()`).

After a partial unregister, the status flips to `Unregistered`, so remaining tokenIds revert on any further unregister, migration, or liquidity calls until admins manual intervention to register any tokenId via `registerCreditForClass()` to set the status back to `Registered` for the credit/class.

Impact

Remaining ERC1155 tokenIds cannot be unregistered, migrated, or have liquidity managed until the credit/class is re-registered, causing temporary denial of service for those tokenIds.

Recommendation

Keep the class level status Registered while any tokenIds remain, and only set it to Unregistered when all tokenIds for the credit/class are removed, or adjust precondition checks to rely on per-tokenId status.

Status

Resolved

[L-02] Multiple Facets - ReentrancyGuard storage collision risk

Description

Multiple facets and contracts inherit from OpenZeppelin's `ReentrancyGuard`, which uses standard storage slot 0 for its internal state variable. This does not follow the diamond storage pattern used throughout the codebase, creating a potential storage collision risk if future facets use standard storage slots.

Vulnerability Details

Multiple facets and contracts inherit from OpenZeppelin's `ReentrancyGuard`:

- `src/contracts/pairs/BaseLpStakingCore.sol`
- `src/facets/staking-manager/KvcmStakingFacet.sol`
- `src/facets/staking-manager/K2StakingFacet.sol`
- `src/facets/reward-manager/RiskyYieldFacet.sol`
- `src/facets/reward-manager/K2YieldFacet.sol`
- `src/facets/aam/operations/OperationsFacet.sol`

The `ReentrancyGuard` implementation uses a private `uint256 _status` variable stored at storage slot 0 (standard storage layout). However, the codebase follows the diamond storage pattern for the diamond contracts. All other storage in the codebase uses namespaced storage slots calculated using `keccak256` hashing.

Impact

Future facets or contracts using standard storage slot 0 could collide with the reentrancy guard's internal state, potentially breaking reentrancy protection or causing unexpected behavior in the diamond contract.

Recommendation

Implement a diamond compatible reentrancy guard using a namespaced storage slot following the same pattern as other storage layouts in the codebase. Replace all instances of `ReentrancyGuard` inheritance in the affected facets with this diamond compatible implementation.

Status

Acknowledged

[L-03] RollUpdateFacet#distributeKvcmLockYieldForInterval, K2YieldCurveFacet#pullK2YieldFromEscrow, CarbonLedger#executeMatureForwardCarbon - Missing separate role for automated functions

Description

The functions `distributeKvcmLockYieldForInterval()`, `pullK2YieldFromEscrow()`, and `executeMatureForwardCarbon()` are designed to be executed periodically via automated jobs, but they use high privilege role modifiers creating an unnecessary security risk, as a compromised private key for the automated job would grant an attacker access to many administrative functions beyond the scope of the automated operations.

Vulnerability Details

The `K2YieldCurveFacet.pullK2YieldFromEscrow()` and `RollUpdateFacet.distributeKvcmLockYieldForInterval()` functions allow the `CARBON_SYSTEM_ADMIN` to call this function.

The `CARBON_SYSTEM_ADMIN` role is the highest privilege role in the system, and grants access to critical system functions across multiple contracts, including contract upgrades via UUPS proxy pattern, role management (granting/revoking roles), system wide configuration changes, parameter updates across various protocol facets, and emergency pause functionality.

The `CarbonLedger.executeMatureForwardCarbon()` function allows the `CARBON_LEDGER_ADMIN` and `CARBON_LEDGER_MANAGER` roles to call it. The `CARBON_LEDGER_MANAGER` role grants access to ledger specific administrative functions, including registering and unregistering carbon classes, registering and unregistering carbon credits, managing ledger configurations, and executing maturity operations.

Since these functions are meant to be called daily by automated jobs, the private key used for these transactions must be stored in an automated system. This increases the attack surface, as automated systems are more susceptible to key leakage.

Impact

Using high privilege roles for automated operations increases the potential impact if the automated job's private key is compromised, as an attacker would gain access to all functions associated with the respective role.

Recommendation

Create dedicated roles specifically for automated operations

Status

Resolved

[L-04] CarbonClassVault#matureForwardCarbon - Unbounded Loops Can Hit Gas Limits

Description

`matureForwardCarbon()` iterates over all registered ERC20 credits and all registered ERC1155 credits and their token IDs. In deployments with many registered credits/token IDs, this can become too expensive and revert due to block gas limits.

Vulnerability Details

```
// Process all ERC20 token credits

for (uint256 i = 0; i < tokenManagement.registeredERC20CarbonCredits.length; i++) {

...

// Process all ERC1155 token credits

for (uint256 i = 0; i < tokenManagement.registeredERC1155CarbonCredits.length; i++) {

...
```

Impact

If a vault ends up with lots of registered credits/tokenIds, `matureForwardCarbon()` can become too expensive to fit in a block and just revert from running out of gas. In that case you can't mature that maturity in one call, so forward balances for that maturity won't get moved into the liquid buckets unless batching / split the work / change the approach is added.

Recommendation

Add a batched version so `matureForwardCarbon()` can be completed over multiple txs, or make it "lazy" (move the update into withdraw/other flows so you don't need one big loop), or split it by token/tokenId so each call processes only a small subset instead of iterating the full registry.

Status

Resolved

[L-05] Contracts - Role Delay/Scheduling Not Enforced in Target Contracts

Description

Multiple contracts use `IAccessManager.hasRole(role, caller)` only as a boolean membership check and ignore the returned delay value. If the protocol relies on AccessManager's execution-delay model (timelock-style safety), these code paths do not enforce it.

Vulnerability Details

As an example, CarbonLedger uses `hasRole()` but discards the delay return value:

```
modifier onlyAdminOrManager() {

    address accessManager = getAccessManager();



    (bool hasLedgerAdminRole,) =

           IAccessManager(accessManager).hasRole(CarbonSystemRoleLib.CARBON_LEDGER_ADMIN,
msg.sender);

    (bool hasLedgerManagerRole,) =

         IAccessManager(accessManager).hasRole(CarbonSystemRoleLib.CARBON_LEDGER_MANAGER,
msg.sender);



    if (!hasLedgerAdminRole && !hasLedgerManagerRole) {

        revert Errors.CallerNotAdminOrManager();

    }

    _;

}
```

The documentation mentions execution delays along with the role hierarchy:

```
[COMPONENT: ProtocolAccessManager] Role-based access control system for the entire carbon protocol. Non-upgradeable implementation of OpenZeppelin's AccessManager with role hierarchy and execution delays.
```

Impact

Unless this is a design choice; If non-zero AccessManager delays are configured for sensitive roles/targets, those delays are not applied in these direct-call code paths. Administrative actions guarded by `hasRole()` can be executed immediately by role members, even when a delay is expected.

Recommendation

Either explicitly document and enforce that all delays are `0` and AccessManager is used only for membership checks or enforce AccessManaged checks (e.g., `restricted` / `canCall`) so the delays apply.

Status

Acknowledged

[L-06] CarbonVaultFactory#updateAccessManager - Authority Update Lacks Contract Type Validation

Description

`CarbonVaultFactory.updateAccessManager()` updates the AccessManager authority by calling `_setAuthority(newAccessManager)` directly. The function checks that the new address is non-zero, but it does not verify that the new authority is actually a contract. This can lead to accidental misconfiguration where an EOA (or an unrelated contract) is set as the authority.

Vulnerability Details

`updateAccessManager()` sets the authority directly:

```
function updateAccessManager(address newAccessManager) external onlySystemAdmin {

    if (newAccessManager == address(0)) revert Errors.ZeroAddressNotAllowed();

    _setAuthority(newAccessManager);

    emit AccessManagerUpdated(newAccessManager);

}
```

Impact

If `newAccessManager` is accidentally set to an EOA or an incompatible address, the factory's authorization mechanism can be broken until another privileged action repairs, potentially blocking deployments/upgrades and other admin operations.

Recommendation

Add an explicit contract-code check before updating authority, e.g. `if (newAccessManager.code.length == 0) revert ...;`, and optionally validate that it behaves like an AccessManager (e.g., supports `hasRole` calls) before accepting it.

Status

Resolved

[L-07] CarbonVaultFactory, ProtocolSupplyOracle - Missing Storage Gaps in Upgradeable Contracts

Description

`CarbonVaultFactory` and `ProtocolSupplyOracle` are upgradeable but do not include a reserved storage gap (`uint256[50] private __gap;`). Other upgradeable contracts in the codebase do include gaps.

Vulnerability Details

`CarbonVaultFactory` defines storage variables but no reserved gap:

```
contract CarbonVaultFactory is
    Initializable,
    AccessManagedUpgradeable,
    PausableUpgradeable,
    UUPSUpgradeable,
    ICarbonVaultFactory
{
    using EnumerableSet for EnumerableSet.AddressSet;

    // Implementation registry
    mapping(uint256 => address) private _implementations;
    uint256[] private _availableVersions;
    uint256 private _latestVersion;

    // Vault tracking
    address[] private _deployedVaults;
    mapping(address => VaultInfo) private _vaultInfo;
```

This same pattern also applies to `ProtocolSupplyOracle`

Impact

Future upgrades have less room to safely add state without risking storage layout mistakes. This increases the chance of upgrade bugs (storage collisions / corrupted state) during future iterations.

Recommendation

Add a reserved storage gap at the end of the contract (`uint256[50] private __gap;`) and keep future new storage variables appended above it.

Status

Resolved

QA

[Q-01] KvcmLockNFT#initialize - Unused owner_ Parameter

Description

`KvcmLockNFT.initialize()` requires a non-zero `owner_` parameter but does not use it to set any role. This can mislead deployers into believing the contract owner is being configured at initialization time.

Vulnerability Details

`owner_` is checked but never stored or used:

```
function initialize(string memory name_, string memory symbol_, address owner_, address accessManager_)
    external
    initializer
{
    if (owner_ == address(0)) revert Errors.ZeroAddressNotAllowed();

    __ERC721_init(name_, symbol_);
    __ERC721URIStorage_init();
    __UUPSUpgradeable_init();
    __ReentrancyGuard_init();

    managerConfigLayout().accessManager = accessManager_;
}
```

Impact

The intended admin/owner may not have expected permissions, which can block upgrades (`_authorizeUpgrade`) or admin functions (`setAccessManager`) depending on role assignment.

Recommendation

Either remove the unused `owner_` parameter or apply it explicitly (grant the intended owner role in AccessManager).

Status

Resolved

[Q-02] CarbonVaultFactory - Unused EnumerableSet Import and Using Directive

Description

CarbonVaultFactory declares `using EnumerableSet for EnumerableSet.AddressSet;` but does not use it.

Vulnerability Details

```
import "@openzeppelin/contracts/utils/structs/EnumerableSet.sol";

...

using EnumerableSet for EnumerableSet.AddressSet;
```

Impact

Minor code quality issue.

Recommendation

Remove the unused import and `using` directive.

Status

Resolved

[Q-03] KvcmLockNFT - Transfer Restrictions Comment Does Not Match Implementation

Description

The contract-level comments state transfers are restricted `from`/`to` a trusted address, but the implementation instead gates transfers based on the caller's role (manager role), not `from`/`to` addresses..

Vulnerability Details

Comment implies "`from`/`to` trusted address", but `_isTransferAllowed` only checks manager role:

```
* Transfers are only allowed to and from a designated trusted address (e.g., staking contract)
```

```
function _isTransferAllowed(address from, address to) internal view returns (bool) {
    // Allow minting and burning (already handled in _update)
    if (from == address(0) || to == address(0)) return true;

    // Only allow transfers if called by liquidity manager
    return _isLiquidityManager();
}
```

Impact

Documentation mismatch can lead to incorrect operational assumptions and incorrect integration constraints around who can move lock NFTs and under what conditions.

Recommendation

Either update the comments to reflect the actual caller role gated transfers policy, or change the logic to enforce the documented trusted address semantics (only allow transfers where `from == stakingManager` or `to == stakingManager`).

Status

Resolved

[Q-04] ProtocolAccessManager#constructor - Redundant zero address check

Description

The `ProtocolAccessManager` constructor performs a redundant zero address validation that is already handled by the parent `AccessManager` constructor.

Vulnerability Details

The `ProtocolAccessManager` constructor checks if `initialAdmin` is the zero address and reverts. However, the parent `AccessManager` constructor already performs the same validation before the child constructor body executes:

```
// src/contracts/ProtocolAccessManager.sol

constructor(address initialAdmin) AccessManager(initialAdmin) {

    if (initialAdmin == address(0)) {

        revert("AccessManagerInvalidInitialAdmin");

    }

}

// src/contracts/access/manager/AccessManager.sol

constructor(address initialAdmin) {

    if (initialAdmin == address(0)) {

        revert AccessManagerInvalidInitialAdmin(address(0));

    }

    // ...

}
```

Impact

The redundant check adds unnecessary code without providing any additional protection, as the parent constructor already enforces this validation.

Recommendation

Remove the redundant zero address check from the `ProtocolAccessManager` constructor.

Status

Resolved

[Q-05] Multiple contracts - Floating pragma

Description

Multiple contracts use floating pragma statements `pragma solidity ^0.8.28;` instead of fixed version pragmas.

Vulnerability Details

Floating pragmas allow contracts to be compiled with different compiler versions than those tested, potentially introducing unidentified security issues.

Impact

Different pragma versions in test and mainnet may pose unidentified security issues.

Recommendation

Consider locking the pragma version to the specific version `pragma solidity 0.8.28;`.

Status

Resolved

[Q-06] MaturityManager#getMaturityTimeframe - Missing maturity ID validation

Description

The `getMaturityTimeframe()` function does not validate whether the provided `maturityId` is valid before accessing the `maturityTimestamps` mapping, which can return incorrect values for invalid maturity IDs.

Vulnerability Details

The `getMaturityTimeframe()` function directly accesses the `maturityTimestamps` mapping without verifying that the provided `maturityId` is within the valid range.

The contract maintains a `currentMaxMaturityId` variable that tracks the maximum valid maturity ID. When an invalid `maturityId` (greater than `currentMaxMaturityId`) is provided, the function will access an uninitialized or zero value in the mapping, resulting in incorrect timestamp calculations.

Other functions in the contract, such as `getMaturityInfo()`, properly validate the maturity ID. The `getMaturityTimeframe()` function should similarly validate that `maturityId <= currentMaxMaturityId` before accessing the mapping.

Impact

Callers of `getMaturityTimeframe()` may receive incorrect timestamp values when providing invalid maturity IDs, potentially leading to incorrect calculations or logic errors in dependent contracts.

Recommendation

Add validation to check that the provided `maturityId` is valid before accessing the mapping. Revert with `Errors.InvalidMaturityId(maturityId)` if the maturity ID exceeds `currentMaxMaturityId`.

Status

Resolved

[Q-07] RMKVCMStakersRewardsFacet#burnSharesForKVCMStaker - Inconsistent Pausability on Burn Functions

Description

Two overloaded `burnSharesForKVCMStaker()` functions have inconsistent pausability modifiers, where one function can be executed when the system is paused while the other cannot.

Vulnerability Details

The contract `RMKVCMStakersRewardsFacet` contains two overloaded `burnSharesForKVCMStaker()` functions that perform similar operations but have different access control modifiers.

`burnSharesForKVCMStaker(uint256 maturityId, address user, address to)` function only has the `onlyStateUpdater` modifier and lacks the `whenNotPaused` modifier, allowing it to be called even when the system is paused.

`burnSharesForKVCMStaker(uint256 maturityId, address user)` function has both `onlyStateUpdater` and `whenNotPaused` modifiers, preventing execution when the system is paused.

Impact

The inconsistent pausability modifiers create ambiguity about whether burn operations should be allowed during system pauses, potentially leading to unexpected behavior and making the contract's pause mechanism less effective.

Recommendation

Add the `whenNotPaused` modifier to `burnSharesForKVCMStaker(uint256 maturityId, address user, address to)` to ensure consistent behavior across both functions.

Status

Resolved

[Q-08] Multiple contracts - State changes without events

Description

Several functions modify state variables but do not emit events, making it difficult for off-chain indexers and monitoring systems to track these important state changes.

Vulnerability Details

The following functions perform state changes without emitting corresponding events:

- `RMKVCMStakersRewardsFacet.mintSharesForKVCMStaker()`
- `CarbonClassVault.updateCreditConfig()`
- `CarbonClassVault.setRegistrationManager()`
- `CarbonLedger.pauseCarbonLedger()`
- `CarbonLedger.unpauseCarbonLedger()`
- `ProtocolRewardsEscrow.setEscrowLimits()`
- `KvcmLockNFT.setAccessManager()`

Impact

Off-chain indexers and monitoring systems cannot efficiently track these state changes, making it difficult to maintain accurate off-chain state, build analytics dashboards, and monitor protocol activity in real-time.

Recommendation

Emit appropriate events for each state changing function.

Status

Resolved

[Q-09] RateLimiterLib#updateLiquidSupplyRateLimiter - Comment Does Not Match Condition

Description

In `RateLimiterLib.updateLiquidSupplyRateLimiter()`, the comment states the rate limiter should be updated when `preTxnLiquidSupply` is greater than or equal to the current effective liquid supply, but the implementation only updates when it is strictly greater than.

Vulnerability Details

The code uses a strict `>` comparison while the comment describes a `>=` check:

```
// if rawLiquidSupply is greater than or equal to effectiveLiquidSupply, update the rate limiter

if (params.preTxnLiquidSupply > currentEffectiveLiquidSupply) {
```

Impact

If the intended behavior was `>=`, then the rate limiter will not update on the equality boundary case.

Recommendation

Either update the comment to match the code (`>`), or change the condition to `>=` if the equality case should trigger a rate limiter update.

Status

Resolved

Centralisation

The KilmaDAO project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the KilmaDAO project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au